✅ Django Middleware – Clean Technical Notes

📌 What is Middleware?

- Middleware is a lightweight, low-level **plugin system** in Django.
- It allows you to globally **intercept, process, and modify**:
  - Incoming **requests**
  - Outgoing **responses**
  - Exceptions
  - Template responses

🧱 Middleware Structure (New Style Only)

- Middleware is a **callable factory** that takes a `get_response` callable.
- It returns another callable that takes a `request` and returns a `response`.

🧠 Basic Flow

1. `__init__(self, get_response)` – runs **once** when the server starts.
2. `__call__(self, request)` – runs **on every request**.
3. Inside `__call__`, call `response = self.get_response(request)` to pass control to the next middleware or view.
4. Code before `get_response()` = **pre-processing** (e.g., modify request).
5. Code after `get_response()` = **post-processing** (e.g., modify response).

✅ Required Method

- `__call__(self, request)`
  Handles each request and returns a response.

✅ Optional Hooks (Only for class-based middleware)

Method	Purpose
`process_view()`	Runs **before the view** is called. Can short-circuit the view.
`process_exception()`	Runs if the **view raises an exception**. Can return a custom response.
`process_template_response()`	Runs if the view returns a `TemplateResponse`. Modify context/templates.

🧬 Middleware Order

- Request phase = **top-down** (in order listed in `MIDDLEWARE`)
- Response phase = **bottom-up** (reverse order)

It's like an onion — each middleware wraps the next one.

⚠️ Short-circuiting

- If a middleware returns a response **without** calling `get_response`, it **short-circuits** the stack.
- Only outer middleware will process the response.

🔐 Activating Middleware

In `settings.py`:

```
MIDDLEWARE = [
"django.middleware.security.SecurityMiddleware",
"django.contrib.sessions.middleware.SessionMiddleware", ...
"your_app.middleware.YourMiddleware", ]
```

- Order matters due to dependencies (e.g., Auth needs Session).
- Can be empty (`MIDDLEWARE = []`) but not recommended.

🌐 Async & Sync Support

Django supports:

Type	How to Support It
Sync only	Default behavior
Async only	Set `async_capable = True`, `sync_capable = False`
Both	Use `@sync_and_async_middleware` from `django.utils.`

- Middleware is adapted by Django to match the view mode (sync/async), but mismatches cause performance hits.
- Use `asgiref.sync.iscoroutinefunction()` to detect async mode.

🧵 Dealing with Streaming Responses

- `StreamingHttpResponse` has no `.content`
- Check with `if response.streaming:` and wrap `streaming_content` generator if needed

Example:

```
if response.streaming: response.streaming_content =
wrap_streaming_content(response.streaming_content) else:
response.content = alter_content(response.content)
```

🚫 Exception Handling

- Middleware doesn't need to catch exceptions from deeper layers.
- Django converts exceptions to `HttpResponse` objects **between** middleware layers.
- You get a response, even if the view raised something.

🧪 Middleware Return Values

Method	Return Type	Effect
`__call__()`	`HttpResponse`	Required
`process_view()`	`HttpResponse` or `None`	Skip view if response is returned
`process_exception()`	`HttpResponse` or `None`	Handle exception if response is returned
`process_template_response`	`TemplateResponse`	Modify or replace the response